

Two ways to automate with Ansible

Ansible can run automation through one-off commands or reusable YAML files.

1 Command method

Ansible ad-hoc commands

Best for quick, simple tasks that do not require a reusable workflow.

2 File method

Ansible playbooks

Best for organized, repeatable automation that must scale and remain maintainable.

Limitations of ad-hoc commands

- You must type the command again whenever you repeat the task.
- Large automation jobs require several separate commands.
- Complex conditions are difficult to express and maintain.
- Repeated input, such as creating many users, needs separate commands.
- Variables and reusable structure are limited compared with playbooks.

Benefits of Ansible playbooks

- Save tasks in a file instead of retyping every command.
- Combine many automation tasks in one organized playbook.
- Apply conditional logic when actions depend on system state.
- Handle multiple inputs and multiple tasks in one workflow.
- Use variables to make automation flexible and reusable.

Ansible playbook basics

- A playbook contains tasks that Ansible performs on managed nodes.
- Ansible playbooks use YAML syntax.
- A playbook can be stored anywhere the user can access it.
- The filename may vary, but the extension should be .yml or .yaml.
- YAML keeps playbooks readable and easy to review.

playbook.yml

```
---  
- name: Install and start  
  Apache  
  hosts: webservers  
  become: true  
  tasks:  
    - name: Install httpd  
      dnf:  
        name: httpd  
        state: present
```

YAML and Ansible playbooks

YAML provides a human-readable way to describe configuration and automation.

- Many references expand YAML as Yet Another Markup Language.
- The official recursive name is YAML Ain't Markup Language.
- YAML represents structured data used by configuration tools.
- Ansible uses YAML for playbooks and related data files.
- AWS CloudFormation and Kubernetes also use YAML documents.

Core YAML writing rules

- A YAML document may begin with --- and may end with ...
- List items start with a hyphen followed by a space.
- Indent related items consistently. YAML does not use tabs for indentation.
- Key-value data uses a colon followed by a space: key: value

Common data forms

String • **List** • **Mapping**

YAML structure

```
---  
- name: Install packages  
  hosts: webservers  
  tasks:  
    - name: Install httpd  
      dnf:  
        name: httpd  
        state: present  
...
```

YAML string format

A string example stores one value under one key.

Single key-value pairs

```
Name: Khalid  
hosts: webserver  
user: root  
package: httpd
```

Personalized example

Name **Khalid**

Format **key: value**

YAML array and list format

A list stores several values under the same key. Every item begins with a hyphen and a space.

Package list

```
package:  
  - httpd  
  - vsftpd  
  - firewalld
```

Indentation note

Keep all list items at the same indentation level. A consistent style makes the YAML easier to read and prevents syntax errors.

YAML mapping format

A mapping groups related subkeys and values under one parent key.

Personal profile mapping

```
Khalid:  
  Phone: +17732222333  
  Address: Chicago  
  Skills:  
    - Linux  
    - Java  
    - Flutter
```

Structure

Parent key	Khalid
Subkeys	Phone, Address, Skills
Rule	Indent child data consistently

From YAML to an Ansible playbook

YAML provides the structure. Ansible keywords and modules define the automation.

YAML

Indentation
Lists
Mappings

Play

Hosts
Privileges
Task list

Tasks

Modules
Arguments
Desired state

A play targets hosts. Each task calls a module with arguments.

Playbook structure

nginx.yml

```
---
- name: Configure web servers
  hosts: web
  become: true

  tasks:
    - name: Install Nginx
      dnf:
        name: nginx
        state: present
```

Play

One automation unit that targets a host pattern.

Task

One named action inside the tasks list.

Module

The Ansible tool that performs the action, such as dnf.

Arguments

Values passed to the module, such as name and state.

First Nginx playbook

nginx.yml

```
---
- name: Install and start Nginx
  hosts: web
  become: true

  tasks:
    - name: Install Nginx
      dnf:
        name: nginx
        state: present

    - name: Start and enable Nginx
      service:
        name: nginx
        state: started
        enabled: true
```

Result

- Targets the web inventory group
- Uses sudo through become
- Installs the nginx package
- Starts nginx and enables it at boot

Play-level keywords

```
---
```

Optional YAML document marker

```
- name
```

A readable description of the play

```
hosts: web
```

Runs the play against the web inventory group

```
become: true
```

Uses privilege escalation for administrative tasks

```
tasks:
```

Starts the ordered list of tasks

Task, module, and arguments

One task

```
- name: Install Nginx
  dnf:
    name: nginx
    state: present
```

- name describes what the task should accomplish.
- dnf selects the package-management module.
- name: nginx identifies the package.
- state: present ensures the package is installed.

Desired state

Ansible compares the current system with the requested state and changes only what is necessary.

Playbook validation

Run checks before making changes to managed nodes.

1. Syntax check

```
ansible-playbook --syntax-check  
nginx.yml
```

Checks YAML and playbook syntax. It does not contact the managed nodes for normal task execution.

2. Check mode

```
ansible-playbook nginx.yml --check
```

Predicts many changes without applying them. Some modules cannot fully simulate every operation.

Optional host check

```
ansible-playbook nginx.yml --list-hosts
```

Running the playbook

Run on every host matched by hosts: web

```
ansible-playbook nginx.yml
```

Run only on node1 within the selected host pattern

```
ansible-playbook nginx.yml --limit node1
```

Use --limit carefully. The limited host must already belong to the play's host pattern.

Idempotency

A well-written playbook can run repeatedly without making unnecessary changes.

First run

changed=2

Nginx is installed and started.

Second run

changed=0

The requested state already exists.

Verification after the playbook

Check the service

```
ansible web -m command \  
  -a "systemctl is-active nginx"
```

Expected service result

active

Check the web response

```
ansible web -m uri \  
  -a "url=http://localhost  
    status_code=200"
```

Expected HTTP result

status: 200

Verification confirms the service state and checks that Nginx answers an HTTP request.

Common playbook mistakes

- Using tabs or inconsistent indentation
- Forgetting the space after a colon
- Targeting an inventory group that does not exist
- Placing module arguments at the wrong indentation level
- Forgetting become: true for package and service administration
- Using a module that does not match the managed node's operating system

Recommended workflow

- 1. Check syntax**
- 2. Review target hosts**
- 3. Use check mode**
- 4. Run the playbook**
- 5. Verify the result**